Lecture 26.04: Linear Regression with Gradient Descent — Step 2: Mean Squared Error (MSE) Cost Function

📌 Executive Summary

Mathematical derivation and geometric analysis of the Mean Squared Error (MSE) cost function, error residuals, convexity guarantees, and vectorized NumPy evaluation.

🔬 Mathematical Formulation & Theory

1. Residual Error Formulation

For any training instance $(\mathbf{x}^{(i)}, y^{(i)})$, the error residual represents the scalar discrepancy between the model's hypothesis prediction $\hat{y}^{(i)}$ and the true observed target $y^{(i)}$:

$$e^{(i)} = \hat{y}^{(i)} - y^{(i)} = (\mathbf{w}^T \mathbf{x}^{(i)} + b) - y^{(i)}$$

2. The Mean Squared Error (MSE) Objective

The aggregate cost over the entire dataset of m training samples is quantified by the Mean Squared Error objective function $J(\mathbf{w}, b)$:

$$J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

3. Why Halve the Mean Squared Error ($rac12m$)?

In classical statistics, MSE is defined with $\frac{1}{m}$. In machine learning optimization, we introduce the factor $\frac{1}{2}$: When taking the partial derivative with respect to any parameter using the power rule:

$$\frac{\partial}{\partial z}\left(\frac{1}{2}z^2\right) = \frac{1}{2} \cdot 2z = z$$

The scalar 2 cancels out perfectly with $\frac{1}{2}$, preventing redundant numerical constants from cluttering the update equations:

$$\nabla_\theta J = \frac{1}{m} X^T (\hat{\mathbf{y}} - \mathbf{y})$$

4. Vectorized Matrix Computation

Expressed in matrix-vector notation:

$$J(\mathbf{w}, b) = \frac{1}{2m} \| X\mathbf{w} + b\mathbf{1}_m - \mathbf{y} \|_2^2 = \frac{1}{2m} (\hat{\mathbf{y}} - \mathbf{y})^T (\hat{\mathbf{y}} - \mathbf{y})$$

```
# Exact one-line vectorized loss computation
residuals = y_pred - y
loss = (1.0 / (2.0 * m)) * np.dot(residuals, residuals)
```

💻 Python & NumPy Implementation

```
import numpy as np

def compute_mse_cost(X, y, weights, bias):
    """Compute vectorized Mean Squared Error cost J(w, b)."""
    m = len(y)
    y_pred = X @ weights + bias
    residuals = y_pred - y
    cost = (1.0 / (2.0 * m)) * np.sum(residuals ** 2)
    return cost

# Test cost computation
X_demo = np.array([[1.0], [2.0], [3.0], [4.0]])
y_demo = np.array([2.0, 4.0, 6.0, 8.0])

w_good = np.array([2.0])
b_good = 0.0

w_bad = np.array([0.5])
```

```
b_bad = 1.0

print(f"Optimal Parameters Cost: {compute_mse_cost(X_demo, y_demo, w_good, b_good)
print(f"Sub-optimal Parameters Cost: {compute_mse_cost(X_demo, y_demo, w_bad, b_ba
```

🎯 Key Takeaways & Best Practices

1. **First-Principles Transparency**: Building from scratch demystifies optimizer mechanics, memory layouts, and algorithmic failure modes.
2. **Vectorization vs Loops**: Always eliminate Python `for` loops across sample dimensions by relying on BLAS-accelerated NumPy array operations.
3. **Shape Integrity**: Maintain explicit assertions on input array ranks and shapes (`X.shape == (m, d)` , `y.shape == (m,)`) to prevent silent broadcasting bugs.